


```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1]
```

```
tf.keras.layers.Dense(64, activation='relu'),
tf.keras.layers.Dense(32, activation='relu'),
tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
```

```

..
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,152
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

465/465 ————— 3s 3ms/step - loss: 1.2238 - mae: 0.7635 - val_1

Epoch 2/100

465/465 ————— 1s 2ms/step - loss: 0.3791 - mae: 0.4364 - val_1

Epoch 3/100

465/465 ————— 1s 3ms/step - loss: 0.3666 - mae: 0.4252 - val_1

Epoch 4/100

465/465 ————— 2s 4ms/step - loss: 0.3285 - mae: 0.4033 - val_1

Epoch 5/100

465/465 ————— 1s 3ms/step - loss: 0.3111 - mae: 0.3911 - val_1

Epoch 6/100

465/465 ————— 1s 2ms/step - loss: 0.3091 - mae: 0.3873 - val_1

Epoch 7/100

465/465 ————— 1s 2ms/step - loss: 0.2953 - mae: 0.3791 - val_1

Epoch 8/100

```

465/465 ----- 1s 2ms/step - loss: 0.2871 - mae: 0.3691 - val_1
Epoch 9/100
465/465 ----- 1s 2ms/step - loss: 0.2876 - mae: 0.3722 - val_1
Epoch 10/100
465/465 ----- 1s 2ms/step - loss: 0.2729 - mae: 0.3614 - val_1
Epoch 11/100
465/465 ----- 1s 2ms/step - loss: 0.2801 - mae: 0.3646 - val_1
Epoch 12/100
465/465 ----- 1s 2ms/step - loss: 0.2834 - mae: 0.3550 - val_1
Epoch 13/100
465/465 ----- 1s 2ms/step - loss: 0.2717 - mae: 0.3606 - val_1
Epoch 14/100
465/465 ----- 1s 2ms/step - loss: 0.2610 - mae: 0.3498 - val_1
Epoch 15/100
465/465 ----- 1s 3ms/step - loss: 0.2612 - mae: 0.3498 - val_1
Epoch 16/100
465/465 ----- 2s 4ms/step - loss: 0.2617 - mae: 0.3481 - val_1
Epoch 17/100
465/465 ----- 1s 2ms/step - loss: 0.2535 - mae: 0.3445 - val_1
Epoch 18/100
465/465 ----- 1s 2ms/step - loss: 0.2486 - mae: 0.3447 - val_1
Epoch 19/100
465/465 ----- 1s 2ms/step - loss: 0.2519 - mae: 0.3418 - val_1
Epoch 20/100
465/465 ----- 1s 2ms/step - loss: 0.2435 - mae: 0.3365 - val_1
Epoch 21/100
465/465 ----- 1s 2ms/step - loss: 0.2497 - mae: 0.3404 - val_1
Epoch 22/100
465/465 ----- 1s 2ms/step - loss: 0.2474 - mae: 0.3369 - val_1
Epoch 23/100
465/465 ----- 1s 2ms/step - loss: 0.2444 - mae: 0.3369 - val_1
Epoch 24/100
465/465 ----- 1s 2ms/step - loss: 0.2444 - mae: 0.3388 - val_1
Epoch 25/100
465/465 ----- 1s 2ms/step - loss: 0.2338 - mae: 0.3314 - val_1
Epoch 26/100
465/465 ----- 1s 2ms/step - loss: 0.2284 - mae: 0.3266 - val_1
Epoch 27/100
465/465 ----- 2s 4ms/step - loss: 0.2390 - mae: 0.3318 - val_1
Epoch 28/100
465/465 ----- 2s 2ms/step - loss: 0.2349 - mae: 0.3291 - val_1
Epoch 29/100
465/465 ----- 1s 2ms/step - loss: 0.2233 - mae: 0.3230 - val_1
Epoch 30/100
465/465 ----- 1s 2ms/step - loss: 0.2289 - mae: 0.3271 - val_1
Epoch 31/100
465/465 ----- 1s 2ms/step - loss: 0.2290 - mae: 0.3288 - val_1
Epoch 32/100
465/465 ----- 1s 2ms/step - loss: 0.2234 - mae: 0.3241 - val_1
Epoch 33/100
465/465 ----- 1s 2ms/step - loss: 0.2251 - mae: 0.3210 - val_1
Epoch 34/100
465/465 ----- 1s 2ms/step - loss: 0.2233 - mae: 0.3232 - val_1
Epoch 35/100

```

```

465/465 ————— 1s 2ms/step - loss: 0.2211 - mae: 0.3205 - val_1
Epoch 36/100
465/465 ————— 1s 2ms/step - loss: 0.2246 - mae: 0.3221 - val_1
Epoch 37/100
465/465 ————— 2s 4ms/step - loss: 0.2126 - mae: 0.3168 - val_1
Epoch 38/100
465/465 ————— 2s 4ms/step - loss: 0.2220 - mae: 0.3217 - val_1
Epoch 39/100
465/465 ————— 1s 2ms/step - loss: 0.2048 - mae: 0.3103 - val_1
Epoch 40/100
465/465 ————— 1s 2ms/step - loss: 0.2257 - mae: 0.3171 - val_1
Epoch 41/100
465/465 ————— 1s 3ms/step - loss: 0.2079 - mae: 0.3138 - val_1
Epoch 42/100
465/465 ————— 1s 2ms/step - loss: 0.2054 - mae: 0.3104 - val_1
Epoch 43/100
465/465 ————— 1s 2ms/step - loss: 0.2025 - mae: 0.3092 - val_1
Epoch 44/100
465/465 ————— 1s 2ms/step - loss: 0.2022 - mae: 0.3072 - val_1
Epoch 45/100
465/465 ————— 1s 2ms/step - loss: 0.2097 - mae: 0.3140 - val_1
Epoch 46/100
465/465 ————— 1s 2ms/step - loss: 0.2092 - mae: 0.3121 - val_1
Epoch 47/100
465/465 ————— 1s 3ms/step - loss: 0.2142 - mae: 0.3138 - val_1
Epoch 48/100
465/465 ————— 2s 4ms/step - loss: 0.2021 - mae: 0.3070 - val_1
Epoch 49/100
465/465 ————— 1s 3ms/step - loss: 0.1988 - mae: 0.3040 - val_1
Epoch 50/100
465/465 ————— 1s 2ms/step - loss: 0.2003 - mae: 0.3059 - val_1
Epoch 51/100
465/465 ————— 1s 2ms/step - loss: 0.1942 - mae: 0.3025 - val_1
Epoch 52/100
465/465 ————— 1s 2ms/step - loss: 0.1959 - mae: 0.3050 - val_1
Epoch 53/100
465/465 ————— 1s 2ms/step - loss: 0.1955 - mae: 0.3009 - val_1
Epoch 54/100
465/465 ————— 1s 2ms/step - loss: 0.2033 - mae: 0.3122 - val_1
Epoch 55/100
465/465 ————— 1s 2ms/step - loss: 0.1962 - mae: 0.3038 - val_1
Epoch 56/100
465/465 ————— 1s 2ms/step - loss: 0.1923 - mae: 0.2993 - val_1
Epoch 57/100
465/465 ————— 1s 2ms/step - loss: 0.1950 - mae: 0.3044 - val_1
Epoch 58/100
465/465 ————— 1s 2ms/step - loss: 0.1867 - mae: 0.2992 - val_1
Epoch 59/100
465/465 ————— 2s 4ms/step - loss: 0.1891 - mae: 0.3000 - val_1
Epoch 60/100
465/465 ————— 1s 3ms/step - loss: 0.1915 - mae: 0.3009 - val_1
Epoch 61/100
465/465 ————— 1s 2ms/step - loss: 0.1828 - mae: 0.2951 - val_1
Epoch 62/100

```

```
465/465 ————— 1s 2ms/step - loss: 0.1843 - mae: 0.2961 - val_1
Epoch 63/100
465/465 ————— 1s 2ms/step - loss: 0.1858 - mae: 0.2990 - val_1
Epoch 64/100
465/465 ————— 1s 2ms/step - loss: 0.1891 - mae: 0.2980 - val_1
Epoch 65/100
465/465 ————— 1s 2ms/step - loss: 0.1939 - mae: 0.3026 - val_1
Epoch 66/100
465/465 ————— 1s 2ms/step - loss: 0.1791 - mae: 0.2939 - val_1
Epoch 67/100
465/465 ————— 1s 2ms/step - loss: 0.1843 - mae: 0.2959 - val_1
Epoch 68/100
465/465 ————— 1s 2ms/step - loss: 0.1807 - mae: 0.2930 - val_1
Epoch 69/100
465/465 ————— 1s 2ms/step - loss: 0.1809 - mae: 0.2929 - val_1
Epoch 70/100
465/465 ————— 2s 4ms/step - loss: 0.1787 - mae: 0.2917 - val_1
Epoch 71/100
465/465 ————— 1s 3ms/step - loss: 0.1774 - mae: 0.2895 - val_1
Epoch 72/100
465/465 ————— 1s 2ms/step - loss: 0.1720 - mae: 0.2883 - val_1
Epoch 73/100
465/465 ————— 1s 2ms/step - loss: 0.1725 - mae: 0.2870 - val_1
Epoch 74/100
465/465 ————— 1s 3ms/step - loss: 0.1792 - mae: 0.2923 - val_1
Epoch 75/100
465/465 ————— 2s 3ms/step - loss: 0.1723 - mae: 0.2850 - val_1
Epoch 76/100
465/465 ————— 1s 3ms/step - loss: 0.1798 - mae: 0.2921 - val_1
Epoch 77/100
465/465 ————— 2s 2ms/step - loss: 0.1818 - mae: 0.2945 - val_1
Epoch 78/100
465/465 ————— 1s 2ms/step - loss: 0.1651 - mae: 0.2837 - val_1
Epoch 79/100
465/465 ————— 2s 4ms/step - loss: 0.1766 - mae: 0.2907 - val_1
Epoch 80/100
465/465 ————— 2s 3ms/step - loss: 0.1735 - mae: 0.2894 - val_1
Epoch 81/100
465/465 ————— 1s 2ms/step - loss: 0.1712 - mae: 0.2874 - val_1
Epoch 82/100
465/465 ————— 1s 2ms/step - loss: 0.1656 - mae: 0.2837 - val_1
Epoch 83/100
465/465 ————— 1s 2ms/step - loss: 0.1665 - mae: 0.2851 - val_1
Epoch 84/100
465/465 ————— 1s 2ms/step - loss: 0.1667 - mae: 0.2804 - val_1
Epoch 85/100
465/465 ————— 1s 2ms/step - loss: 0.1654 - mae: 0.2811 - val_1
Epoch 86/100
465/465 ————— 1s 2ms/step - loss: 0.1696 - mae: 0.2853 - val_1
Epoch 87/100
465/465 ————— 1s 2ms/step - loss: 0.1640 - mae: 0.2786 - val_1
Epoch 88/100
465/465 ————— 1s 2ms/step - loss: 0.1629 - mae: 0.2776 - val_1
Epoch 89/100
465/465 ————— 1s 2ms/step - loss: 0.1590 - mae: 0.2786 - val_1
```

```

465/465 ————— 1s 2ms/step - loss: 0.1599 - mae: 0.2780 - val_1
Epoch 90/100
465/465 ————— 1s 3ms/step - loss: 0.1652 - mae: 0.2830 - val_1
Epoch 91/100
465/465 ————— 2s 3ms/step - loss: 0.1610 - mae: 0.2779 - val_1
Epoch 92/100
465/465 ————— 1s 3ms/step - loss: 0.1671 - mae: 0.2840 - val_1
Epoch 93/100
465/465 ————— 1s 2ms/step - loss: 0.1608 - mae: 0.2776 - val_1
Epoch 94/100
465/465 ————— 1s 2ms/step - loss: 0.1581 - mae: 0.2768 - val_1
Epoch 95/100
465/465 ————— 1s 2ms/step - loss: 0.1595 - mae: 0.2796 - val_1
Epoch 96/100
465/465 ————— 1s 2ms/step - loss: 0.1640 - mae: 0.2785 - val_1
Epoch 97/100
465/465 ————— 1s 2ms/step - loss: 0.1581 - mae: 0.2783 - val_1
Epoch 98/100
465/465 ————— 1s 2ms/step - loss: 0.1562 - mae: 0.2736 - val_1
Epoch 99/100
465/465 ————— 1s 2ms/step - loss: 0.1553 - mae: 0.2771 - val_1
Epoch 100/100
465/465 ————— 1s 2ms/step - loss: 0.1641 - mae: 0.2815 - val_1
129/129 ————— 0s 2ms/step - loss: 0.2671 - mae: 0.3509
129/129 ————— 0s 1ms/step

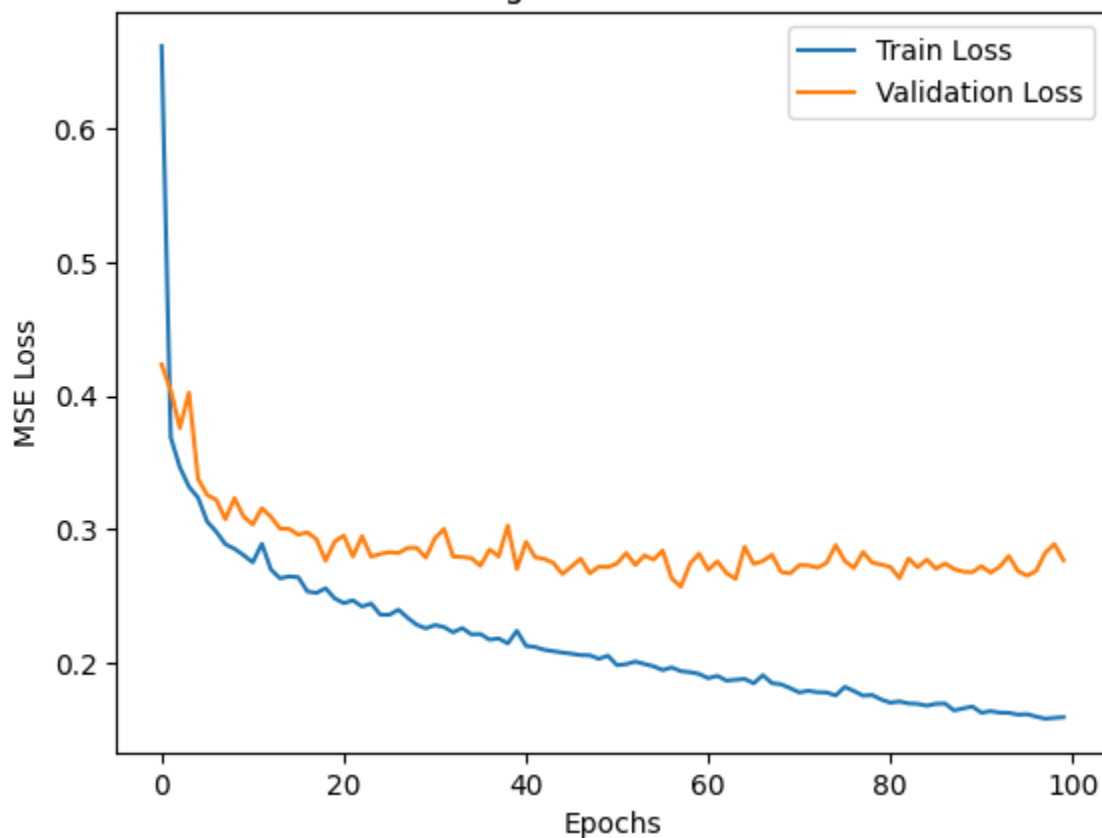
```

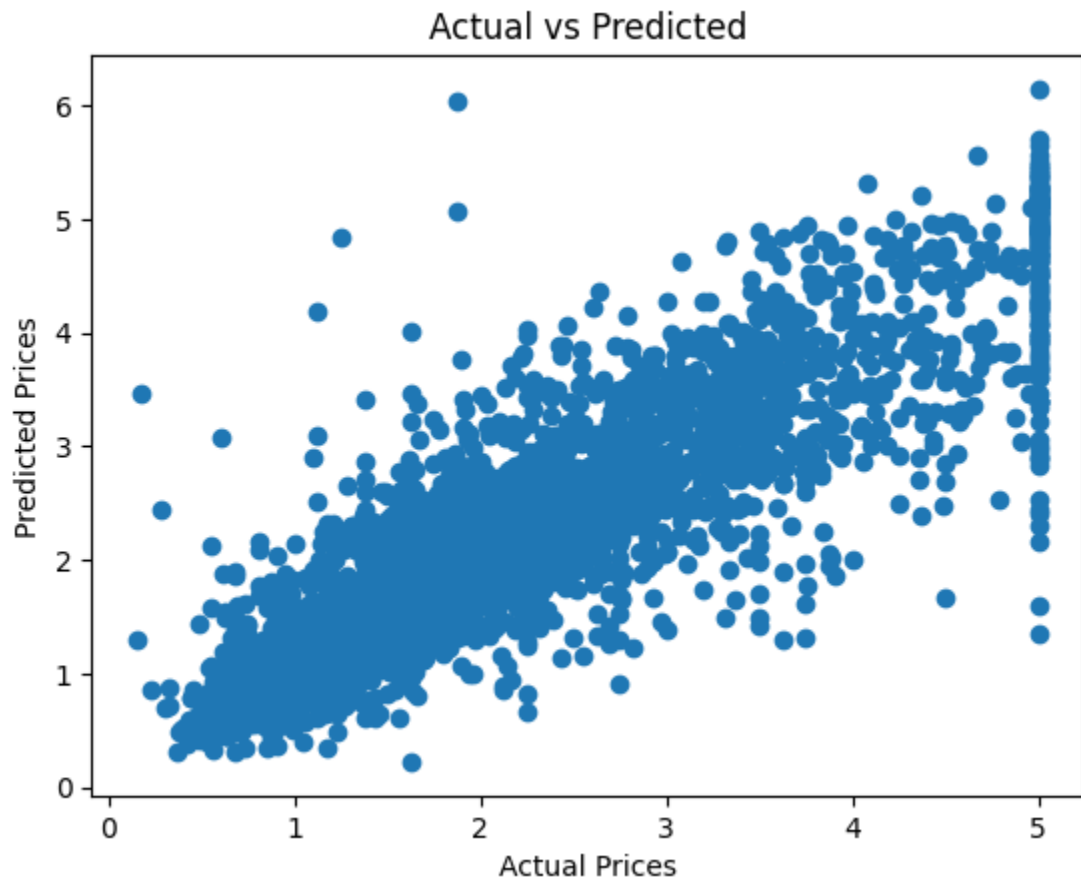
Test MAE: 0.3504222631454468

RMSE: 0.5199928550701585

R2 Score: 0.7936578779496831

Training vs Validation Loss





```
# =====  
# 1. Import Libraries  
# =====  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import tensorflow as tf  
  
from sklearn.datasets import fetch_california_housing  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score  
  
# =====  
# 2. Load Dataset as DataFrame  
# =====  
housing = fetch_california_housing()  
  
df = pd.DataFrame(housing.data, columns=housing.feature_names)  
df["Price"] = housing.target
```

```
print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32, #batch size
    verbose=1
)
```

```

/

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	1,152
dense_5 (Dense)	(None, 64)	8,256
dense_6 (Dense)	(None, 32)	2,080
dense_7 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

523/523 ————— 6s 8ms/step - loss: 1.2594 - mae: 0.7725 - val_1

Epoch 2/100

523/523 ————— 4s 8ms/step - loss: 0.3850 - mae: 0.4426 - val_1

Epoch 3/100

523/523 ————— 3s 6ms/step - loss: 0.3435 - mae: 0.4134 - val_1

Epoch 4/100

523/523 ————— 3s 2ms/step - loss: 0.3319 - mae: 0.4036 - val_1

Epoch 5/100

523/523 ————— 1s 2ms/step - loss: 0.3116 - mae: 0.3873 - val_1

Epoch 6/100

523/523 ————— 1s 2ms/step - loss: 0.3023 - mae: 0.3810 - val_1

Epoch 7/100

523/523 ————— 1s 2ms/step - loss: 0.3024 - mae: 0.3769 - val_1

Epoch 8/100

523/523 ————— 1s 2ms/step - loss: 0.2875 - mae: 0.3711 - val_1

Epoch 9/100

523/523 ————— 2s 4ms/step - loss: 0.2965 - mae: 0.3694 - val_1

Epoch 10/100

523/523 ————— 2s 3ms/step - loss: 0.2765 - mae: 0.3601 - val_1

Epoch 11/100

523/523 ————— 1s 2ms/step - loss: 0.2778 - mae: 0.3624 - val_1

Epoch 12/100

523/523 ————— 1s 2ms/step - loss: 0.2985 - mae: 0.3595 - val_1

Epoch 13/100

523/523 ————— 1s 2ms/step - loss: 0.2616 - mae: 0.3525 - val_1

Epoch 14/100

```
# =====
```

```
# 1. Import Libraries
```

```
# =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from sklearn.datasets import fetch_california_housing
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()
```

```

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8 3252	41 0	6 984177	1 073810	322 0	2 555556	37 88	

```

0  8.3014      21.0  6.238137  0.971880      2401.0  2.109842  37.86
1  7.2574      52.0  8.288136  1.073446      496.0  2.802260  37.85
2  5.6431      52.0  5.817352  1.073059      558.0  2.547945  37.85
3  3.8462      52.0  6.281853  1.081081      565.0  2.181467  37.85

```

```

Longitude Price
0 -122.23 4.526
1 -122.22 3.585
2 -122.24 3.521
3 -122.25 3.413
4 -122.25 3.422

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 128)	1,152
dense_9 (Dense)	(None, 64)	8,256
dense_10 (Dense)	(None, 32)	2,080
dense_11 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

407/407 ————— 5s 7ms/step - loss: 1.3546 - mae: 0.8111 - val_1

Epoch 2/100

407/407 ————— 1s 2ms/step - loss: 0.4031 - mae: 0.4479 - val_1

Epoch 3/100

407/407 ————— 1s 2ms/step - loss: 0.3468 - mae: 0.4201 - val_1

Epoch 4/100

407/407 ————— 1s 2ms/step - loss: 0.3463 - mae: 0.4146 - val_1

Epoch 5/100

407/407 ————— 1s 2ms/step - loss: 0.3263 - mae: 0.3967 - val_1

Epoch 6/100

407/407 ————— 1s 2ms/step - loss: 0.3310 - mae: 0.3971 - val_1

Epoch 7/100

407/407 ————— 1s 2ms/step - loss: 0.3185 - mae: 0.3886 - val_1

Epoch 8/100

407/407 ————— 1s 2ms/step - loss: 0.3164 - mae: 0.3874 - val_1

Epoch 9/100

407/407 ————— 1s 2ms/step - loss: 0.2993 - mae: 0.3765 - val_1

Epoch 10/100

407/407 ————— 1s 2ms/step - loss: 0.2866 - mae: 0.3710 - val_1

Epoch 11/100

407/407 ————— 1s 3ms/step - loss: 0.2861 - mae: 0.3702 - val_1

Epoch 12/100

407/407 ————— 2s 4ms/step - loss: 0.2891 - mae: 0.3676 - val_1

Epoch 13/100

407/407 ————— 1s 3ms/step - loss: 0.2855 - mae: 0.3618 - val_1

Epoch 14/100

```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape
    tf.keras.layers.Dense(64, activation='relu'),
```



```
tf.keras.layers.Dense(32, activation='relu'),
tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=64,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10 Actual vs Predicted Plot
```

```

# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 128)	1,152
dense_13 (Dense)	(None, 64)	8,256
dense_14 (Dense)	(None, 32)	2,080
dense_15 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

233/233 ————— 2s 4ms/step - loss: 1.7485 - mae: 0.9228 - val_1

Epoch 2/100

233/233 ————— 1s 3ms/step - loss: 0.4019 - mae: 0.4470 - val_1

Epoch 3/100

233/233 ————— 1s 3ms/step - loss: 0.3507 - mae: 0.4219 - val_1

Epoch 4/100

233/233 ————— 1s 3ms/step - loss: 0.3428 - mae: 0.4160 - val_1

Epoch 5/100

233/233 ————— 1s 2ms/step - loss: 0.3380 - mae: 0.4044 - val_1

Epoch 6/100

233/233 ————— 1s 4ms/step - loss: 0.3262 - mae: 0.3965 - val_1

Epoch 7/100

233/233 ————— 1s 4ms/step - loss: 0.3102 - mae: 0.3857 - val_1

Epoch 8/100

233/233 ————— 1s 4ms/step - loss: 0.3043 - mae: 0.3846 - val_1

```

Epoch 9/100
233/233 ██████████ 1s 3ms/step - loss: 0.2973 - mae: 0.3777 - val_1
Epoch 10/100
233/233 ██████████ 1s 2ms/step - loss: 0.2918 - mae: 0.3748 - val_1
Epoch 11/100
233/233 ██████████ 1s 3ms/step - loss: 0.2815 - mae: 0.3657 - val_1
Epoch 12/100
233/233 ██████████ 1s 3ms/step - loss: 0.2795 - mae: 0.3632 - val_1
Epoch 13/100
233/233 ██████████ 1s 2ms/step - loss: 0.2772 - mae: 0.3627 - val_1
Epoch 14/100

```

```

# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
..

```

```
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=128,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
```

```
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()
```

```
# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 128)	1,152
dense_17 (Dense)	(None, 64)	8,256
dense_18 (Dense)	(None, 32)	2,080
dense_19 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

117/117 ————— 2s 4ms/step - loss: 2.4951 - mae: 1.1541 - val_1

Epoch 2/100

117/117 ————— 0s 3ms/step - loss: 0.5312 - mae: 0.5155 - val_1

```

Epoch 3/100
117/117 ————— 0s 3ms/step - loss: 0.4201 - mae: 0.4497 - val_1
Epoch 4/100
117/117 ————— 0s 3ms/step - loss: 0.3678 - mae: 0.4313 - val_1
Epoch 5/100
117/117 ————— 0s 3ms/step - loss: 0.3707 - mae: 0.4301 - val_1
Epoch 6/100
117/117 ————— 0s 3ms/step - loss: 0.3413 - mae: 0.4137 - val_1
Epoch 7/100
117/117 ————— 0s 3ms/step - loss: 0.3448 - mae: 0.4162 - val_1
Epoch 8/100
117/117 ————— 0s 3ms/step - loss: 0.3370 - mae: 0.4058 - val_1
Epoch 9/100
117/117 ————— 0s 4ms/step - loss: 0.3182 - mae: 0.3968 - val_1
Epoch 10/100
117/117 ————— 1s 5ms/step - loss: 0.3191 - mae: 0.4009 - val_1
Epoch 11/100
117/117 ————— 1s 5ms/step - loss: 0.3056 - mae: 0.3854 - val_1
Epoch 12/100
117/117 ————— 1s 4ms/step - loss: 0.3089 - mae: 0.3878 - val_1
Epoch 13/100
117/117 ————— 0s 3ms/step - loss: 0.2977 - mae: 0.3774 - val_1
Epoch 14/100

```

```

# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

```

```
y = y_train

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=256,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)
```

```

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_20 (Dense)	(None, 128)	1,152
dense_21 (Dense)	(None, 64)	8,256

dense_22 (Dense)	(None, 32)	2,080
dense_23 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

59/59  2s 7ms/step - loss: 3.1773 - mae: 1.3747 - val_loss: 3.1773

Epoch 2/100

59/59  0s 4ms/step - loss: 0.7521 - mae: 0.6244 - val_loss: 0.7521

Epoch 3/100

59/59  0s 4ms/step - loss: 0.5323 - mae: 0.5177 - val_loss: 0.5323

Epoch 4/100

59/59  0s 4ms/step - loss: 0.4399 - mae: 0.4704 - val_loss: 0.4399

Epoch 5/100

59/59  0s 4ms/step - loss: 0.3865 - mae: 0.4441 - val_loss: 0.3865

Epoch 6/100

59/59  0s 4ms/step - loss: 0.3620 - mae: 0.4267 - val_loss: 0.3620

Epoch 7/100

59/59  0s 4ms/step - loss: 0.3611 - mae: 0.4236 - val_loss: 0.3611

Epoch 8/100

59/59  0s 4ms/step - loss: 0.3433 - mae: 0.4160 - val_loss: 0.3433

Epoch 9/100

59/59  0s 4ms/step - loss: 0.3476 - mae: 0.4149 - val_loss: 0.3476

Epoch 10/100

59/59  0s 5ms/step - loss: 0.3324 - mae: 0.4075 - val_loss: 0.3324

Epoch 11/100

59/59  0s 4ms/step - loss: 0.3260 - mae: 0.4029 - val_loss: 0.3260

Epoch 12/100

59/59  0s 4ms/step - loss: 0.3188 - mae: 0.3959 - val_loss: 0.3188

Epoch 13/100

59/59  0s 4ms/step - loss: 0.3122 - mae: 0.3920 - val_loss: 0.3122

Epoch 14/100

```
# =====
# 1. Import Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()
```

```
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
```

```

        epochs=100,
        batch_size=512,
        verbose=1
    )

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()

```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585

```

1      122.22  3.500
2      -122.24  3.521
3      -122.25  3.413
4      -122.25  3.422

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_24 (Dense)	(None, 128)	1,152
dense_25 (Dense)	(None, 64)	8,256
dense_26 (Dense)	(None, 32)	2,080
dense_27 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

30/30 ————— 2s 15ms/step - loss: 4.1939 - mae: 1.6637 - val_lo

Epoch 2/100

30/30 ————— 0s 6ms/step - loss: 0.9506 - mae: 0.7136 - val_lo

Epoch 3/100

30/30 ————— 0s 7ms/step - loss: 0.6755 - mae: 0.5915 - val_lo

Epoch 4/100

30/30 ————— 0s 6ms/step - loss: 0.5453 - mae: 0.5306 - val_lo

Epoch 5/100

30/30 ————— 0s 6ms/step - loss: 0.4535 - mae: 0.4848 - val_lo

Epoch 6/100

30/30 ————— 0s 5ms/step - loss: 0.4157 - mae: 0.4606 - val_lo

Epoch 7/100

30/30 ————— 0s 6ms/step - loss: 0.3930 - mae: 0.4424 - val_lo

Epoch 8/100

30/30 ————— 0s 6ms/step - loss: 0.3872 - mae: 0.4472 - val_lo

Epoch 9/100

30/30 ————— 0s 6ms/step - loss: 0.3578 - mae: 0.4251 - val_lo

Epoch 10/100

30/30 ————— 0s 5ms/step - loss: 0.3682 - mae: 0.4257 - val_lo

Epoch 11/100

30/30 ————— 0s 5ms/step - loss: 0.3686 - mae: 0.4258 - val_lo

Epoch 12/100

30/30 ————— 0s 6ms/step - loss: 0.3523 - mae: 0.4191 - val_lo

Epoch 13/100

30/30 ————— 0s 6ms/step - loss: 0.3356 - mae: 0.4108 - val_lo

Epoch 14/100

```
# =====
```

```
# 1. Import Libraries
```

```
# =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

```

```
    metrics=[mae]
)

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=1024,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_7"

Layer (type)	Output Shape	Param #
dense_28 (Dense)	(None, 128)	1,152
dense_29 (Dense)	(None, 64)	8,256
dense_30 (Dense)	(None, 32)	2,080
dense_31 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

15/15 ————— 4s 39ms/step - loss: 5.0287 - mae: 1.9329 - val_lo

Epoch 2/100

15/15 ————— 1s 18ms/step - loss: 1.8954 - mae: 0.9730 - val_lo

Epoch 3/100

15/15 ————— 0s 18ms/step - loss: 1.0368 - mae: 0.7356 - val_lo

Epoch 4/100

15/15 ————— 0s 9ms/step - loss: 0.7985 - mae: 0.6424 - val_lo

Epoch 5/100

15/15 ————— 0s 10ms/step - loss: 0.7292 - mae: 0.6143 - val_lo

Epoch 6/100

15/15 ————— 0s 11ms/step - loss: 0.6481 - mae: 0.5772 - val_lo

Epoch 7/100

15/15 ————— 0s 10ms/step - loss: 0.5672 - mae: 0.5389 - val_lo

Epoch 8/100

15/15 ————— 0s 12ms/step - loss: 0.5076 - mae: 0.5103 - val_lo

Epoch 9/100

15/15 ————— 0s 10ms/step - loss: 0.4682 - mae: 0.4888 - val_lo

Epoch 10/100

15/15 ————— 0s 11ms/step - loss: 0.4305 - mae: 0.4681 - val_lo

Epoch 11/100

15/15 ————— 0s 10ms/step - loss: 0.4116 - mae: 0.4599 - val_lo

Epoch 12/100

15/15 ————— 0s 9ms/sten - loss: 0.3970 - mae: 0.4489 - val los

```
Epoch 13/100
15/15  0s 9ms/step - loss: 0.3748 - mae: 0.4379 - val_loss
Epoch 14/100
15/15  0s 9ms/step - loss: 0.3788 - mae: 0.4411 - val_loss
Epoch 15/100
15/15  0s 9ms/step - loss: 0.3674 - mae: 0.4320 - val_loss
Epoch 16/100
15/15  0s 10ms/step - loss: 0.3653 - mae: 0.4317 - val_loss
Epoch 17/100
15/15  0s 9ms/step - loss: 0.3567 - mae: 0.4254 - val_loss
Epoch 18/100
15/15  0s 13ms/step - loss: 0.3572 - mae: 0.4227 - val_loss
Epoch 19/100
15/15  0s 9ms/step - loss: 0.3552 - mae: 0.4231 - val_loss
Epoch 20/100
15/15  0s 10ms/step - loss: 0.3445 - mae: 0.4152 - val_loss
Epoch 21/100
15/15  0s 9ms/step - loss: 0.3369 - mae: 0.4105 - val_loss
Epoch 22/100
15/15  0s 10ms/step - loss: 0.3487 - mae: 0.4172 - val_loss
Epoch 23/100
15/15  0s 10ms/step - loss: 0.3424 - mae: 0.4121 - val_loss
Epoch 24/100
15/15  0s 11ms/step - loss: 0.3270 - mae: 0.4034 - val_loss
Epoch 25/100
15/15  0s 10ms/step - loss: 0.3286 - mae: 0.4033 - val_loss
```